

Lexical & Semantic Retrieval on MIND and EB-NeRD

CS4.406 Information Retrieval & Extraction — Assignment 1 Design Note | Shubham Paliwal |
github.com/Shubham-749/IRE-Assignment1

1. Overview

Built a reproducible pipeline for both MIND (English) and EB-NeRD (Danish) covering: a unified data pipeline with temporal (never random) splitting and a leakage-safe point-in-time history accessor (Q1); lexical (BM25) and semantic (embedding) candidate generation (Q2/Q3); an offline evaluation harness with the official ranking metrics, beyond-accuracy metrics, and bootstrap confidence intervals (Q4); and real Codabench submissions, including one real, evidence-based improvement iteration on MIND (Q5). Every number below is measured against the real downloaded datasets and real leaderboard scores, not simulated.

2. Pipeline and Design Choices (Q1–Q3)

Data pipeline & leakage boundary.

Both datasets already ship official time-ordered train/val folders. Rather than re-split randomly — which would leak future clicks into training, the opposite of how a deployed recommender sees data — the pipeline validates that the official folders are genuinely temporally ordered (`verify_split_integrity`) and reuses them directly. User history lives in one long-format table, and every retrieval method reads it through a single point-in-time accessor (`get_user_history` / `UserHistoryIndex`) that always filters to clicks strictly before the impression being scored — one testable leakage boundary (Q9) instead of logic duplicated into every feature.

Lexical retrieval (BM25).

A dict-based inverted index and classic Okapi BM25 ($k_1=1.5$, $b=0.75$) over title+abstract, built from scratch rather than a library — simple and transparent at this scale (12K–125K articles), and it let the same scoring formula be reused unchanged for both full-corpus retrieval (Q2) and impression-level candidate scoring (Q4).

Semantic retrieval (embeddings).

A multilingual sentence-transformer (paraphrase-multilingual-MiniLM-L12-v2) so one model covers both English MIND and Danish EB-NeRD, with brute-force cosine similarity (L2-normalized matrix + matvec) rather than FAISS — simpler at 12K–125K articles and explicitly allowed by the assignment; FAISS is the natural next step at 10x+ scale (§5). A user's query vector is the mean-pooled, re-normalized embedding of their recent click history. **Alternative rejected:** lightweight TF-IDF+SVD embeddings (no heavy dependency) — rejected because it is still built on the same bag-of-words counts as BM25 and unlikely to show genuine semantic generalization the way a real contextual model can.

3. Evaluation Harness (Q4)

AUC (tie-aware Mann-Whitney rank formula), MRR, and $nDCG@5,10$ implemented from scratch, plus diversity / novelty / coverage, all with bootstrap 95% confidence intervals, and a cold-start (<5 history items) vs. warm slice. This operates at the *impression* level — ranking the small candidate list Codabench itself scores — a genuinely different task from Q2/Q3's full-corpus candidate-generation recall, and both are reported because they can disagree (§4).

4. Key Findings: Lexical vs. Semantic, and Where They Disagree

Dataset	Method	recall@50	recall@100	recall@200	AUC (Q4)	Diversity@5
MIND-small	BM25	0.53%	1.15%	2.19%	0.553	0.917
MIND-small	Embeddings	0.88%	1.43%	2.27%	0.633	0.847
EB-NeRD demo	BM25	0.83%	1.55%	2.76%	0.494	0.842
EB-NeRD demo	Embeddings	0.56%	1.12%	2.78%	0.539	0.780

The two datasets disagree, and that disagreement is the finding, not noise. **On MIND, embeddings win decisively** at every K and every ranking metric — BM25's impression-level AUC (0.553) is barely above the 0.5 chance level. **On EB-NeRD, BM25 keeps a real recall edge** at tight budgets, and at the impression-ranking level embeddings' AUC (0.494 offline) is itself indistinguishable from chance. Spot-checking examples explains why: BM25 finds articles that lexically echo a user's history, but real clicks are often on something topically adjacent with zero word overlap — the classic lexical-retrieval gap embeddings exist to close. On EB-NeRD, Danish headlines are terse and formulaic enough that exact keyword match stays a strong signal on its own, so semantic expressiveness does not pay off the same way. Accuracy and diversity trade off directly on both datasets too: the more-accurate embedding ranking is consistently the less-diverse one (tighter thematic clustering) — a beyond-accuracy metric surfacing a tradeoff AUC alone never would.

5. Codabench Submissions and Iterating on Real Results (Q5)

BM25 was not used for either real submission, and this was measured, not assumed: timed at 11–170ms/impression, it would take on the order of **days** across MIND's 2.37M or EB-NeRD's 13.5M impressions. Embedding similarity vectorizes (batched matrix ops) and is the only method actually tractable at real scale.

MIND: a real, evidence-based improvement.

First submission (AUC 0.6194, MRR 0.3006, nDCG@5 0.3225, nDCG@10 0.3784) used the shared multilingual embedding model — a real compromise, since MIND is 100% English and a multilingual model trades per-language quality for cross-lingual coverage MIND never needed. Hypothesis: an English-specific model (all-mpnet-base-v2) would help. Validated cheaply offline on MIND-small *before* committing to the expensive full-catalog re-embed: AUC 0.6513 vs. 0.6333, 95% confidence intervals non-overlapping ([0.643,0.659] vs. [0.625,0.642]) — a real signal, not sampling noise. Re-submitted and confirmed on the live leaderboard: **AUC 0.6194 → 0.6544**, MRR 0.3006 → 0.3211, nDCG@5 0.3225 → 0.3464, nDCG@10 0.3784 → 0.4023 — every official metric improved, not just the one the hypothesis targeted.

901007	mind_prediction.zip	2026-08-25 14:15	Finished	0.6544	✓ ↻
900737	mind_prediction.zip	2026-08-25 10:49	Finished	0.6194	📄 ↻ 🗑

Fig. 1 — Both MIND submissions: 0.6194 (original) and 0.6544 (after the model swap).

34	shubham749	2026-08-25 14:15	901007	0.6544	0.3211	0.3464	0.4023
----	------------	------------------	--------	--------	--------	--------	--------

Fig. 2 — Improved submission's full leaderboard row (rank shown for reference only; grading is never on leaderboard rank).

EB-NeRD: the same discipline, an honest negative result.

The identical lever does not transfer. Since Danish genuinely needs multilingual support, the analogous move was upgrading model *capacity* within the multilingual family (MiniLM→mpnet-base). Tested offline first, exactly as for MIND — and the result was flat-to-negative (AUC 0.539→0.529, MRR and nDCG also down), consistent with §4's finding that EB-NeRD's task is more lexical than semantic: a larger embedding model does not fix a bottleneck it was never the cause of. The expensive full-scale EB-NeRD resubmission was correctly *not* attempted on the strength of this result. Reporting a negative finding honestly, and not spending compute chasing it further, is itself the point of validating offline first.

EB-NeRD submission status.

13,536,710 predictions generated and submitted, fully validated pre-upload (exact expected count, correct sequential order matching the source file, correct required filename inside the zip). Codabench's own scoring stage has been stuck on "Running" since submission; course staff confirmed this is a known platform-side issue currently under investigation with the organizers, not a problem with the submission itself, and advised keeping the zip file safe for when scoring resumes. The result will be reported if/when the platform recovers.

ID #	File name	Date	Status	Score	Detailed results	Actions
902112	predictions.zip	2026-08-26 08:55	Submitted			📄
901797	predictions.zip	2026-08-26 01:47	Submitted			📄

Fig. 3 — EB-NeRD submissions on Codabench: accepted ("Submitted"), no score yet — the platform-side scoring stall described above.

Testing and format verification.

48 tests cover BM25/embedding correctness, metric formulas against hand-computed examples, leakage boundaries, and download idempotency. Two of them exist because a real submission failed twice, not because we anticipated the bugs: Codabench rejected the first MIND upload for the zip containing the wrong internal filename (mind_prediction.txt instead of the platform's hardcoded expected prediction.txt), then rejected the fix for scrambled row order — sorting by a string impression_id column put "10" right after "1" lexicographically, but Codabench compares predictions to ground truth *positionally*, line by line, in the original file's order. Both fixes were centralized into one shared, tested function (rank_and_group) used by *both* datasets' submission scripts, so EB-NeRD's pipeline never had to rediscover either bug independently.

6. Where This Pipeline Breaks at 10x Scale

- **BM25 does not scale without re-architecting.** Measured, not hypothetical: at 11–170ms/impression it would take days at real Codabench volumes. A batched/vectorized sparse-matrix BM25 or a real search engine (Lucene/tantivy) would be needed at 10x+.
- **Brute-force embedding search needs an ANN index at 10x+.** Fine at 12K–125K articles, but matvec cost grows linearly with catalog size and is already the majority of per-query cost at 125K articles — FAISS is the natural next step.
- **Memory, not just CPU, is the real ceiling at 10x, and it showed up for real.** Generating EB-NeRD's actual submission (13.5M impressions, 807,677 users) repeatedly crashed a 17.2GB development machine. Root cause: a helper built for leakage-safe point-in-time filtering was reused for a task that never needed that filtering, silently exploding 807K users' history into 116.8M rows just to deduplicate and immediately re-aggregate it back down. Fixed by reading raw per-user history directly. The most instructive lesson of the assignment: code correct and well-tested at small scale can still hide a real cost that only appears once production-scale data arrives — and initial profiling suspicion (the scoring loop) was itself wrong, even with real data and real crashes in front of us.
- **Single-machine, single-process throughout.** No distributed compute anywhere in this pipeline. Real production scale (all users, daily retraining) would need horizontal scaling (Spark/Ray for the data pipeline, a served ANN index rather than an in-process matrix) that nothing here currently provides.

7. Anti-Gaming and Leakage (Q9)

Every user-history read goes through the one tested point-in-time accessor described in §2; a dedicated test suite (`test_no_leakage.py`) asserts no click ever appears in a user's history before it happened, run against the real downloaded data, not synthetic fixtures alone. One documented data limitation, not a workaround for convenience: MIND provides no per-click timestamps, only order, so synthetic per-user timestamps are assigned that are guaranteed to sit strictly before that split's earliest real impression.

Ablation: metrics with vs. without a serving-time-unavailable feature.

`scripts/run_leakage_ablation.py` deliberately reintroduces one concrete leaked feature — each user's clicks from their *other* validation-split impressions, appended to their query on top of the real point-in-time history — and re-runs Q4's exact harness with it. The impression's own answer is excluded from its own leak set (using the literal label would be a trivial ablation). The no-leak column below reproduces Q4's published numbers exactly, confirming the harness reuse is correct, not a different sample.

Dataset	Method	AUC: no-leak → leak	MRR: no-leak → leak
MIND-small	BM25	0.5529 → 0.5526	0.2934 → 0.2816
MIND-small	Embeddings	0.6333 → 0.6316	0.3402 → 0.3367
EB-NeRD demo	BM25	0.4944 → 0.4890	0.3122 → 0.2973
EB-NeRD demo	Embeddings	0.5391 → 0.5434	0.3384 → 0.3431

The leak mostly does not help: 3 of 4 dataset/method combinations get slightly *worse*, not better, since the appended titles come from a user's clicks on unrelated other impressions and mostly dilute the query rather than pointing at the right answer. EB-NeRD embeddings is the one exception (a small, consistent gain across all four metrics), plausibly because a user's other same-session clicks broaden the mean-pooled taste vector in a genuinely relevant direction. Worth stating plainly: even a real, non-trivial serving-time-unavailable signal does not automatically inflate these metrics. Full numbers (all four metrics, all bootstrap CIs) are in `results/leakage_ablation.csv`.